

Android 导航栏（Navigation Bar）使用详解

Android 导航栏（Navigation Bar）使用详解

Android 导航栏分为**系统原生导航栏**（底部虚拟按键：返回、主页、多任务）和**应用内导航组件**（如 Jetpack Navigation、第三方导航库），以下分两部分讲解：

一、系统原生导航栏（系统级）

系统导航栏是 Android 系统提供的全局导航控件，默认在屏幕底部，核心作用是全局页面切换，开发者可对其进行**样式定制**或**行为控制**，但无法直接修改核心功能（返回/主页等）。

1. 核心特性

- 系统默认提供：返回键（←）、主页键（○）、多任务键（⊕）（部分机型可自定义）；
- 沉浸式/全屏模式下可隐藏，滑动边缘呼出；
- 适配不同屏幕（折叠屏、平板）的布局。

2. 开发者常用操作

（1）隐藏/显示导航栏（沉浸式）

需申请权限并通过 Window 控制，示例：

Code block

```
1  // 隐藏导航栏（沉浸式，需配合隐藏状态栏）
2  val decorView = window.decorView
3  val uiOptions = View.SYSTEM_UI_FLAG_HIDE_NAVIGATION or
    View.SYSTEM_UI_FLAG_FULLSCREEN
4  decorView.systemUiVisibility = uiOptions
5
6  // 监听导航栏显示/隐藏（避免用户滑动呼出后无法恢复）
7  decorView.setOnSystemUiVisibilityChangeListener { visibility ->
8      if (visibility and View.SYSTEM_UI_FLAG_HIDE_NAVIGATION == 0) {
9          // 导航栏显示了，可重新隐藏
10         decorView.systemUiVisibility = uiOptions
11     }
12 }
```

（2）修改导航栏样式（颜色/对比度）

在 `res/values/styles.xml` 中配置：

Code block

```
1 <style name="AppTheme" parent="Theme.MaterialComponents.DayNight">
2     <!-- 导航栏背景色 -->
3     <item name="android:navigationBarColor">@color/black</item>
4     <!-- 导航栏图标颜色 (仅 API 26+) -->
5     <item name="android:navigationBarIconColor">@color/white</item>
6     <!-- 适配深色导航栏的文字对比度 (API 27+) -->
7     <item name="android:enforceNavigationBarContrast">false</item>
8 </style>
```

(3) 强制使用手势导航 (API 30+)

仅引导用户设置，无法直接修改系统默认导航模式：

Code block

```
1 // 跳转到系统导航设置页面
2 val intent = Intent(Settings.ACTION_NAVIGATION_MODE_SETTINGS)
3 startActivity(intent)
```

二、应用内导航组件（重点）

应用内导航是指**应用内部页面/Fragment**之间的切换，Android 官方推荐 Jetpack Navigation，同时也有成熟的第三方库。

1. 官方 Jetpack Navigation（核心）

Jetpack Navigation 是 Google 推出的标准化导航组件，解决 Fragment 切换、回退栈管理、深层链接等问题，核心优势：

- 可视化配置 (Navigation Graph) ；
- 自动管理回退栈；
- 支持安全的参数传递；
- 适配深色模式、多窗口等。

核心使用步骤：

(1) 添加依赖 (build.gradle.kts)

Code block

```
1 dependencies {
2     // 核心库
3     implementation("androidx.navigation:navigation-fragment-ktx:2.7.7")
4     implementation("androidx.navigation:navigation-ui-ktx:2.7.7")
5 }
```

(2) 创建 Navigation Graph (导航图)

在 `res/navigation` 目录下新建 `nav_graph.xml`，可视化配置页面/Fragment 关系：

Code block

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <navigation xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     android:id="@+id/nav_graph"
5     app:startDestination="@id/homeFragment">
6
7     <!-- 首页 Fragment -->
8     <fragment
9         android:id="@+id/homeFragment"
10        android:name="com.example.HomeFragment"
11        android:label="首页">
12        <!-- 跳转到详情页的动作 -->
13        <action
14            android:id="@+id/action_home_to_detail"
15            app:destination="@id/detailFragment"
16            app:enterAnim="@anim/slide_in_right"
17            app:exitAnim="@anim/slide_out_left"/>
18    </fragment>
19
20    <!-- 详情页 Fragment -->
21    <fragment
22        android:id="@+id/detailFragment"
23        android:name="com.example.DetailFragment"
24        android:label="详情页">
25        <!-- 接收参数 -->
26        <argument
27            android:name="id"
28            android:defaultValue="-1"
29            app:argType="integer"/>
30    </fragment>
31 </navigation>
```

(3) 在布局中添加 NavHost (导航容器)

```

1 <!-- 布局 -->
2 <androidx.fragment.app.FragmentContainerView
3     android:id="@+id/nav_host_fragment"
4     android:name="androidx.navigation.fragment.NavHostFragment"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     app:defaultNavHost="true"
8     app:navGraph="@navigation/nav_graph"/>

```

(4) 代码中实现导航跳转

Code block

```

1 // HomeFragment 跳转到 DetailFragment (带参数)
2 btnDetail.setOnClickListener {
3     val bundle = bundleOf("id" to 100)
4     findNavController().navigate(R.id.action_home_to_detail, bundle)
5 }
6
7 // DetailFragment 接收参数
8 val id = arguments?.getInt("id", -1)
9
10 // 返回上一页
11 btnBack.setOnClickListener {
12     findNavController().popBackStack()
13 }

```

(5) 结合 BottomNavigationView (底部导航栏)

Code block

```

1 <!-- 底部导航栏 -->
2 <com.google.android.material.bottomnavigation.BottomNavigationView
3     android:id="@+id/bottom_nav"
4     android:layout_width="match_parent"
5     android:layout_height="wrap_content"
6     app:menu="@menu/bottom_nav_menu"/>

```

Code block

```

1 // 绑定导航控制器
2 val navHostFragment =
    supportFragmentManager.findFragmentById(R.id.nav_host_fragment) as
    NavHostFragment
3 val navController = navHostFragment.navController

```

2. 主流第三方导航库

官方 Navigation 满足大部分场景，但部分场景下第三方库更灵活，以下是常用库：

(1) ARouter (阿里)

- **核心优势：**跨模块导航、支持Activity/Fragment/URL 跳转、拦截器、参数自动注入；
- **适用场景：**大型模块化项目；
- **使用示例：**

Code block

```
1  // 1. 添加依赖
2  implementation 'com.alibaba:arouter-api:1.5.2'
3  kapt 'com.alibaba:arouter-compiler:1.5.2'
4
5  // 2. 注解标记页面
6  @Route(path = "/app/detail")
7  class DetailActivity : AppCompatActivity() {
8      @Autowired
9      var id: Int = -1 // 自动注入参数
10     override fun onCreate(savedInstanceState: Bundle?) {
11         super.onCreate(savedInstanceState)
12         ARouter.getInstance().inject(this) // 注入参数
13     }
14 }
15
16 // 3. 跳转
17 ARouter.getInstance()
18     .build("/app/detail")
19     .withInt("id", 100)
20     .navigation()
```

(2) Coordinator (轻量级)

- **核心优势：**极简 API、支持 Fragment/Activity 导航、自定义转场；
- **适用场景：**小型项目、快速开发；

(3) FragmentNavigator (基于官方扩展)

- **核心优势：**解决官方 Navigation 切换 Fragment 重复创建问题、支持懒加载；
- **适用场景：**需要优化 Fragment 性能的场景。

三、关键注意事项

1. **官方优先**：Jetpack Navigation 是 Google 主推，适配性最好，中小型项目首选；
2. **模块化项目**：优先用 ARouter 解决跨模块导航；
3. **性能优化**：避免频繁创建 Fragment，可结合 ViewPager2 + Fragment 懒加载；
4. **回退栈管理**：官方 Navigation 自动管理，第三方库需手动处理；
5. **适配**：导航栏样式需适配深色模式、全面屏（避免被导航栏遮挡内容）。

总结

- 系统导航栏：控制全局系统级导航，仅可定制样式/隐藏，无法修改核心功能；
- 应用内导航：优先用 Jetpack Navigation（标准化、易维护），模块化项目用 ARouter；
- 第三方库：按需选择，核心解决官方库的痛点（如跨模块、性能优化）。

（注：文档部分内容可能由 AI 生成）